

R

Relief

Disaster Insurance Management System

SOFTWARE REQUIREMENTS SPECIFICATION

Document Version:	1.0
Date:	March 2026
Organization:	Hartford India
Prepared By:	Ratna Jashwanth

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Project Overview
- 1.3 Scope
- 1.4 Definitions and Abbreviations
- 1.5 References

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 Assumptions and Dependencies

3. System Architecture

- 3.1 Architectural Overview
- 3.2 Interaction Flow Example
- 3.3 Security Architecture

4. Functional Requirements

- 4.1 Authentication & User Management
- 4.2 Policy Management
- 4.3 Premium Calculation Engine
- 4.4 Claim Management
- 4.5 Disaster Zone Management
- 4.6 Risk Pool Management
- 4.7 Admin Dashboard & Settings

5. Claim Processing Lifecycle

6. Non-Functional Requirements

- 6.1 Security
- 6.2 Performance
- 6.3 Reliability & Availability
- 6.4 Maintainability & Scalability
- 6.5 Usability

7. Database Design

- 7.1 Entity Overview
- 7.2 Key Relationships

8. External Interface Requirements: API

- 8.1 Standard Response Envelope
- 8.2 Endpoint Summary
- 8.3 Error Codes

9. UML & Workflow Diagrams

- 9.1 Use Case Diagram — Policy Request to Premium Paid
- 9.2 Use Case Diagram — Claim Request to Claim Paid
- 9.3 Use Case Diagram — Create Risk Pool and Disaster Zone
- 9.4 ER Diagram
- 9.5 Activity Diagram — Pay Premium Workflow
- 9.6 Sequence Diagram — Pay Premium

10. Technical Challenges & Solutions

11. Future Enhancements

12. Conclusion

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for **Relief**, a web-based Disaster Insurance Management System. It serves as the authoritative reference for developers, testers, architects, and stakeholders throughout the project lifecycle. The document captures the system's intended behaviour, constraints, user roles, business logic engines, data design, and interface specifications, ensuring all parties share a single, unambiguous understanding of what the system must do and how it must perform.

1.2 Project Overview

Relief is a full-stack, role-driven digital platform designed to automate the entire lifecycle of disaster insurance — from policy application through claim settlement. It replaces fragmented manual workflows with a centralised system covering customer onboarding, policy underwriting, premium collection, claim processing, and risk pool management, delivered via a secure Spring Boot REST API consumed by an Angular SPA.

1.3 Scope

The system covers the following functional domains:

- User registration, authentication, and role-based access control
- Policy application, underwriting, approval, and activation
- Dynamic premium calculation using disaster zone risk factors
- Multi-stage claim submission, review, and financial adjudication
- Payment processing for premiums and claim payouts
- Disaster zone configuration and risk assessment
- Risk pool financial monitoring and health tracking
- Admin analytics dashboards across all roles

Out of scope for v1.0: Real payment gateway integration, email/SMS notifications, document upload, and third-party disaster data feeds.

1.4 Definitions and Abbreviations

Term / Abbreviation	Definition
SRS	Software Requirements Specification
JWT	JSON Web Token — stateless authentication mechanism (HS256)
RBAC	Role-Based Access Control
ORM	Object-Relational Mapper (JPA / Hibernate)
DTO	Data Transfer Object
SPA	Single Page Application
Risk Factor	Numerical score (0–10) representing disaster probability and severity
Risk Pool	Financial reserve per disaster type funded by premiums to cover claim payouts
Sum Insured	Maximum payable amount under a policy
Premium	Amount paid by policyholder to activate coverage
Underwriting	Process of evaluating and pricing an insurance application
PENDING	Initial status of a newly submitted policy
ACTIVE	Status of a policy after premium is paid

Term / Abbreviation	Definition
FILED	Initial status of a submitted claim

1.5 References

- Spring Boot Documentation v4.0.3 — <https://spring.io>
- Angular Documentation v21 — <https://angular.io>
- Spring Security Reference — <https://docs.spring.io/spring-security>
- JWT Specification RFC 7519 — <https://tools.ietf.org/html/rfc7519>
- IEEE Std 830-1998: Recommended Practice for SRS

2. Overall Description

2.1 Product Perspective

Relief is a standalone, decoupled web application. The Angular frontend communicates exclusively with the Spring Boot backend via REST APIs secured by JWT. The system integrates no external services in v1.0. All payment logic, risk calculations, and claim adjudication are self-contained. The system is deployable on-premise or in cloud environments (AWS, Azure, GCP) with minimal configuration changes.

2.2 Product Functions Summary

#	Function	Primary Actor(s)
1	User registration and secure login	All roles
2	Policy application and lifecycle tracking	Customer
3	Policy underwriting and premium adjustment	Agent
4	Policy final approval and administration	Admin
5	Dynamic premium calculation using risk factors	System
6	Claim submission and status tracking	Customer
7	Claim review and financial adjudication	Claims Officer
8	Disaster zone configuration and risk management	Admin
9	Risk pool monitoring and health tracking	Admin
10	Role-specific analytics dashboards	All roles

2.3 User Classes and Characteristics

Role	Description	Technical Expertise
Customer	Registers, applies for policies, pays premiums, files claims	Low — consumer-friendly UI required
Agent	Reviews and underwrites policy applications for assigned regions	Medium — insurance domain knowledge
Claims Officer	Investigates and adjudicates all system-wide claims	High — financial decision maker
Admin	Manages users, zones, risk pools, roles, and policy approvals	High — system administrator

2.4 Operating Environment

Component	Technology / Environment
Backend Framework	Spring Boot 4.0.3 (Java 21)
Frontend Framework	Angular 21 (TypeScript), Single Page Application
Database	H2 (development); PostgreSQL / MySQL (production)
ORM	Hibernate / Spring Data JPA
Authentication	JSON Web Tokens (JWT), HS256 algorithm
UI Styling	Tailwind CSS 4.1 + Angular Material
Build Tools	Maven 3.8+ (backend), npm 9+ / Angular CLI 21+ (frontend)
Browsers Supported	Chrome 90+, Firefox 88+, Edge 90+, Safari 14+
Deployment	On-premise or cloud (AWS / Azure / GCP)
Min. Client Resolution	1280 × 720 (optimised for 1920 × 1080)

2.5 Design and Implementation Constraints

- All API endpoints (except /login and /register) must require a valid JWT Bearer token
- Passwords must be hashed with BCrypt (min. 10 rounds) — plain text storage is prohibited
- Sum insured must never exceed property value; validated at both frontend and backend
- Claim filing is only permitted for policies in ACTIVE status
- Premium payment is only permitted for policies in APPROVED status
- The system must enforce valid state-machine transitions at the service layer
- All monetary calculations must use decimal types to prevent floating-point errors
- One pending claim per policy is permitted at any time
- HTTPS with TLS 1.2+ is required for all production deployments

2.6 Assumptions and Dependencies

- Users have access to a modern web browser with JavaScript enabled
- System runs on HTTPS in production; JWT tokens must not be sent over plain HTTP
- Payment processing is simulated in v1.0 — no real financial transactions occur
- Initial disaster zone and risk pool configurations are provided by domain experts
- Risk factor calculations have been validated by licensed actuaries
- Single currency (USD) and single language (English) for initial release

3. System Architecture

3.1 Architectural Overview

Relief is built on a decoupled, multi-tiered RESTful architecture. Each layer has a single, well-defined responsibility, promoting separation of concerns, testability, and maintainability.

Layer	Technology	Responsibility
Presentation	Angular 21 SPA (TypeScript)	Renders role-specific UI, handles client-side validation, consumes REST APIs
API / Routing	Spring Boot Controllers	Routes HTTP requests, validates JWT, enforces RBAC, parses DTOs
Business Logic	Spring Service Layer (Java 21)	Enforces domain rules: premium calculation, state transitions, coverage limit checks
Data Access	Repository Pattern + JPA/Hibernate	Abstracts SQL, manages CRUD, handles entity relationships and transactions
Storage	H2 / PostgreSQL / MySQL	Persists all structured data: users, policies, claims, payments, zones, pools

3.2 Interaction Flow: Customer Applies for Policy (Example)

1. Customer submits policy application form in Angular UI
2. Angular → POST /api/users/{userId}/policies + JWT in Authorization header
3. JwtFilter validates token signature and extracts user role
4. Spring Security verifies CUSTOMER role for the endpoint
5. Controller passes DTO to UserPolicyService
6. Service validates: $\text{sum insured} \leq \text{property value}$, maps disaster zone, calculates risk factor and premium, generates policy number, assigns agent
7. Repository inserts Policy record (status = PENDING)
8. Service returns PolicyResponse DTO
9. Controller returns 201 Created → Angular refreshes UI

3.3 Security Architecture

- JWT tokens are signed with a server-side secret and contain userId and role claims
- All backend endpoints apply Spring Security role-based rules via @PreAuthorize annotations
- The Angular frontend decodes JWT to render role-appropriate navigation — backend is the true security boundary
- Passwords are hashed using BCrypt (10 rounds) before database storage; plain text never persisted
- HTTPS with TLS 1.2+ is required for all production communications
- SQL injection is prevented through JPA parameterised queries — no dynamic SQL construction
- XSS is prevented via Angular's built-in sanitization and Content Security Policy headers

4. Functional Requirements

4.1 Authentication & User Management

ID	Requirement	Actor	Priority
FR-01	System shall allow customers to self-register with name, email, and password	Public	High
FR-02	System shall validate that the email address is unique across all accounts	System	High
FR-03	System shall hash all passwords with BCrypt (10 rounds) before storage	System	High
FR-04	System shall authenticate users via email/password and return a signed JWT with userId and role	Public	High
FR-05	System shall enforce role-based access control on all protected endpoints	System	High
FR-06	Admin shall create accounts for all user types (Customer, Agent, Claims Officer, Admin)	Admin	High
FR-07	Admin shall assign or modify user roles	Admin	High
FR-08	Admin shall activate or deactivate user accounts	Admin	High
FR-09	Authenticated users shall retrieve their own profile from the system	All roles	Medium

4.2 Policy Management

ID	Requirement	Actor	Priority
FR-10	Customer shall apply for policies by specifying disaster type, region, policy type, property details, sum insured, and tenure	Customer	High
FR-11	System shall validate that $\text{sumInsured} \leq \text{property value}$ at the time of submission	System	High
FR-12	System shall map the policy to a disaster zone and calculate premium using: $\text{sumInsured} \times \text{baseRate} \times (\text{riskFactor} / 5.0)$	System	High
FR-13	System shall auto-assign an agent to the policy based on the selected region	System	High
FR-14	Customer shall view all their policies with filtering by status and disaster type	Customer	High
FR-15	Customer shall pay the premium for APPROVED policies to activate them	Customer	High
FR-16	System shall transition policy status to ACTIVE and set start/end dates upon successful premium payment	System	High
FR-17	System shall update the risk pool's totalPremiumCollected upon every premium payment	System	High
FR-18	Agent shall view all policies assigned to their geographic region	Agent	High
FR-19	Agent shall adjust premium amounts with mandatory justification remarks	Agent	High
FR-20	Agent shall forward reviewed policies to Admin for final approval or reject them with a reason	Agent	High
FR-21	Admin shall give final approval or rejection to policies in UNDER_REVIEW status	Admin	High
FR-22	System shall enforce valid policy transitions: PENDING → UNDER_REVIEW → APPROVED → ACTIVE or REJECTED	System	High

4.3 Premium Calculation Engine

Parameter	Rule / Value
Base Formula	$\text{Premium} = \text{sumInsured} \times \text{baseRate} \times (\text{riskFactor} / 5.0)$
Disaster Type Score	FLOOD = 6 EARTHQUAKE = 9 CYCLONE = 8 WILDFIRE = 5
Region Score	NORTH = 4 SOUTH = 7 EAST = 5 WEST = 4 CENTRAL = 6
Risk Factor Formula	$(\text{disasterTypeScore} + \text{regionScore}) / 2$
Risk Level	$< 5.5 = \text{LOW}$ $5.5 - 6.9 = \text{MEDIUM}$ $\geq 7.0 = \text{HIGH}$

4.4 Claim Management

ID	Requirement	Actor	Priority
FR-23	Customer shall file claims against ACTIVE policies with description and estimated loss amount	Customer	High
FR-24	System shall reject claims if the associated policy is not in ACTIVE status	System	High
FR-25	System shall reject a new claim if a pending claim already exists for the same policy	System	High
FR-26	Claims Officer shall mark FILED claims as UNDER_REVIEW to begin formal investigation	Claims Officer	High
FR-27	Claims Officer shall approve claims with a specified payout amount and officer remarks	Claims Officer	High
FR-28	System shall enforce that approved amount $\leq$ policy sum insured AND $\leq$ estimated loss	System	High
FR-29	System shall verify the risk pool has sufficient funds before approving a claim	System	Medium
FR-30	Upon approval, system shall create a CLAIM_PAYOUT payment record and update risk pool's totalClaimsPaid	System	High
FR-31	Claims Officer shall reject claims with a mandatory rejection reason	Claims Officer	High
FR-32	System shall flag claims with estimated loss > \$50,000 for priority review	System	Medium
FR-33	System shall enforce valid transitions: FILED → UNDER_REVIEW → APPROVED / REJECTED	System	High

4.5 Disaster Zone Management

ID	Requirement	Actor	Priority
FR-34	Admin shall create disaster zones with zone name, location, disaster type, risk level, and risk factor (0–10)	Admin	High
FR-35	System shall use the zone's risk factor in premium calculation at policy creation time	System	High
FR-36	Admin shall update zone parameters (risk level, risk factor) to reflect changing real-world conditions	Admin	High
FR-37	Admin shall delete zones only when no active policies are linked to them	Admin	Low

4.6 Risk Pool Management

ID	Requirement	Actor	Priority
FR-38	Admin shall create one risk pool per disaster type, specifying an alert threshold percentage	Admin	High
FR-39	System shall automatically increment totalPremiumCollected on each successful premium payment	System	High
FR-40	System shall automatically increment totalClaimsPaid on each approved claim payout	System	High
FR-41	System shall calculate pool utilisation as $(\text{totalClaimsPaid} / \text{totalPremiumCollected} \times 100)$ and flag pools exceeding threshold	System	High
FR-42	Admin shall view all risk pools with health status indicators: HEALTHY / WARNING / CRITICAL	Admin	High

4.7 Admin Dashboard & Settings

ID	Requirement	Actor	Priority
FR-43	Admin dashboard shall display total users, policies by status, claims by status, premiums collected, claims paid, and pool health	Admin	High
FR-44	Agent dashboard shall display total policies underwritten, pending reviews, approval ratio, and premium collected	Agent	Medium
FR-45	Claims Officer dashboard shall display total claims processed, pending reviews, approved payouts, high-value claims, and rejection rate	Claims Officer	Medium
FR-46	Customer dashboard shall display active policies, pending premiums, and claim statuses	Customer	Medium

5. Claim Processing Lifecycle

Claims follow a strict state machine enforced at the service layer. No transition can be skipped or reversed. Invalid status changes are rejected with HTTP 422.

Current Status	Trigger / Action	Next Status	Actor
—	Customer submits claim against ACTIVE policy	FILED	Customer
FILED	Claims Officer begins formal investigation	UNDER_REVIEW	Claims Officer
UNDER_REVIEW	Claims Officer approves with valid payout amount	APPROVED	Claims Officer
UNDER_REVIEW	Claims Officer rejects with mandatory reason	REJECTED	Claims Officer
APPROVED	System creates CLAIM_PAYOUT payment record	PAID	System
APPROVED / REJECTED	No further transitions permitted	—	—

Business rules enforced during the lifecycle:

- Approved amount must not exceed the policy's sum insured
- Approved amount must not exceed the customer's estimated loss
- Risk pool must have sufficient available funds before claim approval
- Policy must remain ACTIVE throughout the claim processing period
- Only one pending claim (FILED or UNDER_REVIEW) is allowed per policy at a time
- Claims with estimated loss > \$50,000 are automatically flagged for priority review

6. Non-Functional Requirements

6.1 Security

ID	Requirement
NFR-01	All API endpoints except /login and /register must require a valid JWT Bearer token
NFR-02	JWT tokens must be signed with a server-managed secret and must include expiry claims (24-hour lifetime)
NFR-03	Passwords must never be stored in plain text; BCrypt with minimum 10 rounds is mandatory
NFR-04	Role-based access must be enforced at the API controller level, not just the frontend
NFR-05	SQL injection must be prevented through JPA parameterised queries — no dynamic SQL construction
NFR-06	XSS must be prevented via Angular's built-in sanitization and Content Security Policy headers
NFR-07	HTTPS with TLS 1.2+ is required for all production deployments
NFR-08	All security-relevant events (auth attempts, auth failures, data modifications) must be logged

6.2 Performance

ID	Requirement
NFR-09	95th-percentile API response time for standard CRUD endpoints must be under 2 seconds
NFR-10	The premium calculation engine must return a result within 500 ms
NFR-11	The system must support at least 100 concurrent users without performance degradation
NFR-12	Frontend pages must load and become interactive within 3 seconds on a 5 Mbps connection
NFR-13	Database queries must execute within 500 ms; complex joins must remain under 1 second

6.3 Reliability & Availability

ID	Requirement
NFR-14	System must target 99% uptime (max 7.2 hours downtime/month, excluding planned maintenance)
NFR-15	Database transactions must be ACID-compliant; partial writes on failure must be fully rolled back
NFR-16	System must handle invalid or expired JWT tokens gracefully, always returning 401 — never 500
NFR-17	Failed multi-step operations (e.g., pay premium + update risk pool) must roll back atomically

6.4 Maintainability & Scalability

ID	Requirement
NFR-18	Backend must follow a layered architecture — Controllers, Services, and Repositories clearly separated
NFR-19	System must be deployable with environment-variable configuration; no hardcoded secrets in source code
NFR-20	Stateless JWT-based API design must support horizontal scaling without server-side sessions
NFR-21	All public APIs must be documented via Swagger / OpenAPI accessible in development environment
NFR-22	Test coverage must reach a minimum of 70% for critical business logic paths

6.5 Usability

ID	Requirement
NFR-23	Angular frontend must render role-appropriate navigation based on the decoded JWT payload
NFR-24	All forms must provide real-time, field-level validation feedback before submission

ID	Requirement
NFR-25	UI must be responsive across devices from 375 px (mobile) to 1920 px (desktop)
NFR-26	System must comply with WCAG 2.1 Level AA accessibility guidelines
NFR-27	All error messages must be user-friendly, specific, and suggest corrective actions

7. Database Design

The relational schema is normalised to eliminate redundancy, maintain referential integrity, and support accurate historical financial tracking. H2 is used for development; the schema is portable to PostgreSQL and MySQL for production.

7.1 Entity Overview

Table	Primary Key	Key Foreign Keys	Description
users	id (BIGINT)	role_id → roles	All system users across all roles with BCrypt-hashed passwords
roles	id (BIGINT)	—	System roles: CUSTOMER, AGENT, CLAIMS_OFFICER, ADMIN
agents	id (BIGINT)	user_id → users	Agent-specific info: license number and geographic region
policies	id (BIGINT)	user_id, agent_id, disaster_zone_id, risk_pool_id	Insurance contracts with full lifecycle status tracking
claims	id (BIGINT)	policy_id, risk_pool_id, assigned_officer_id	Insurance claims with status, amounts, and payout tracking
payments	id (BIGINT)	policy_id, claim_id, risk_pool_id	All financial transactions — both premiums and payouts
disaster_zones	id (BIGINT)	—	Geographic zones with risk level and calculated risk factor
risk_pools	id (BIGINT)	—	Financial reserves per disaster type, tracking premiums and payouts
claim_audit_logs	id (BIGINT)	claim_id, performed_by → users	Immutable audit trail of all claim status changes
survey_reports	id (BIGINT)	claim_id, surveyor_id → users	Field survey data submitted by surveyors for claims

7.2 Key Relationships

- One Role → Many Users (1:N)
- One User → Many Policies as Customer (1:N)
- One Agent → Many Policies (1:N)
- One DisasterZone → Many Policies (1:N)
- One RiskPool → Many Policies, Claims, and Payments (1:N)
- One Policy → Many Claims (1:N)
- One Policy → One Premium Payment (1:1 per policy term)
- One Claim → One Payout Payment upon approval (1:1)
- One Claim → Many ClaimAuditLogs (1:N)
- One Claim → One SurveyReport (1:1)

8. External Interface Requirements: API

All endpoints communicate over HTTPS using **application/json**. Every protected endpoint requires the header: **Authorization: Bearer <jwt_token>**

8.1 Standard Response Envelope

All API responses conform to the following JSON structure:

```
{ "status": 200, "message": "Optional description", "data": { ... } }
```

8.2 Endpoint Summary

Method	Endpoint	Access	Description
POST	/login	Public	Authenticate user, receive signed JWT
POST	/register	Public	Customer self-registration
GET	/api/users/{id}/dashboard	Customer	Customer analytics dashboard
GET	/api/users/{id}/policies	Customer	List all own policies
POST	/api/users/{id}/policies	Customer	Submit new policy application
GET	/api/users/{id}/policies/{policyId}	Customer	Full policy details
POST	/api/users/{id}/payments/pay-premium	Customer	Pay premium for APPROVED policy
GET	/api/users/{id}/claims	Customer	List all own claims
POST	/api/users/{id}/claims	Customer	File a new claim
GET	/api/users/{id}/claims/{claimId}	Customer	Full claim details
GET	/api/agents/dashboard	Agent	Agent performance dashboard
GET	/api/agents/policies	Agent	List policies assigned to agent
PUT	/api/agents/policies/{id}/adjust-premium	Agent	Adjust premium with justification
PUT	/api/agents/policies/{id}/forward	Agent	Forward policy to Admin for approval
PUT	/api/agents/policies/{id}/reject	Agent	Reject policy with reason
GET	/api/claims-officer/dashboard	Claims Officer	Claims processing dashboard
GET	/api/claims-officer/claims	Claims Officer	List all system claims
PUT	/api/claims-officer/claims/{id}/under-review	Claims Officer	Mark claim as UNDER_REVIEW
PUT	/api/claims-officer/claims/{id}/approve	Claims Officer	Approve claim with payout amount
PUT	/api/claims-officer/claims/{id}/reject	Claims Officer	Reject claim with reason
GET	/api/admin/dashboard	Admin	System-wide executive dashboard
POST	/api/admin/users	Admin	Create new user account
PUT	/api/admin/users/{id}/assign-role	Admin	Assign or change user role
PUT	/api/admin/users/{id}/status	Admin	Activate or deactivate user
POST	/api/admin/agents	Admin	Create agent profile
PUT	/api/admin/policies/{id}/approve	Admin	Final policy approval
PUT	/api/admin/policies/{id}/reject	Admin	Final policy rejection

Method	Endpoint	Access	Description
GET	/api/admin/disaster-zones	Admin	List all disaster zones
POST	/api/admin/disaster-zones	Admin	Create new disaster zone
PUT	/api/admin/disaster-zones/{id}	Admin	Update zone risk parameters
DELETE	/api/admin/disaster-zones/{id}	Admin	Delete zone (no active policies)
GET	/api/admin/risk-pools	Admin	List all risk pools with health status
POST	/api/admin/risk-pools	Admin	Create risk pool for disaster type
PUT	/api/admin/risk-pools/{id}	Admin	Update risk pool parameters

8.3 Error Codes

HTTP Code	Meaning	Common Cause
400	Bad Request	Missing required fields, invalid input, or business rule violation
401	Unauthorized	Missing, expired, or malformed JWT token
403	Forbidden	Valid token but insufficient role permissions
404	Not Found	Requested resource does not exist in the system
409	Conflict	Duplicate record (e.g., email already registered, pool already exists for disaster type)
422	Unprocessable Entity	Data is syntactically valid but fails business logic (e.g., sum insured > property value)
500	Internal Server Error	Unexpected server-side failure

9. UML & Workflow Diagrams

The following diagrams provide a comprehensive visual specification of the Relief system, covering actors, workflows, data models, and interaction sequences.

9.1 Use Case Diagram — Policy Request to Premium Paid

Illustrates the end-to-end flow from policy application submission through agent underwriting, admin approval, premium payment, and policy activation. Actors involved: Customer, Admin, Agent.

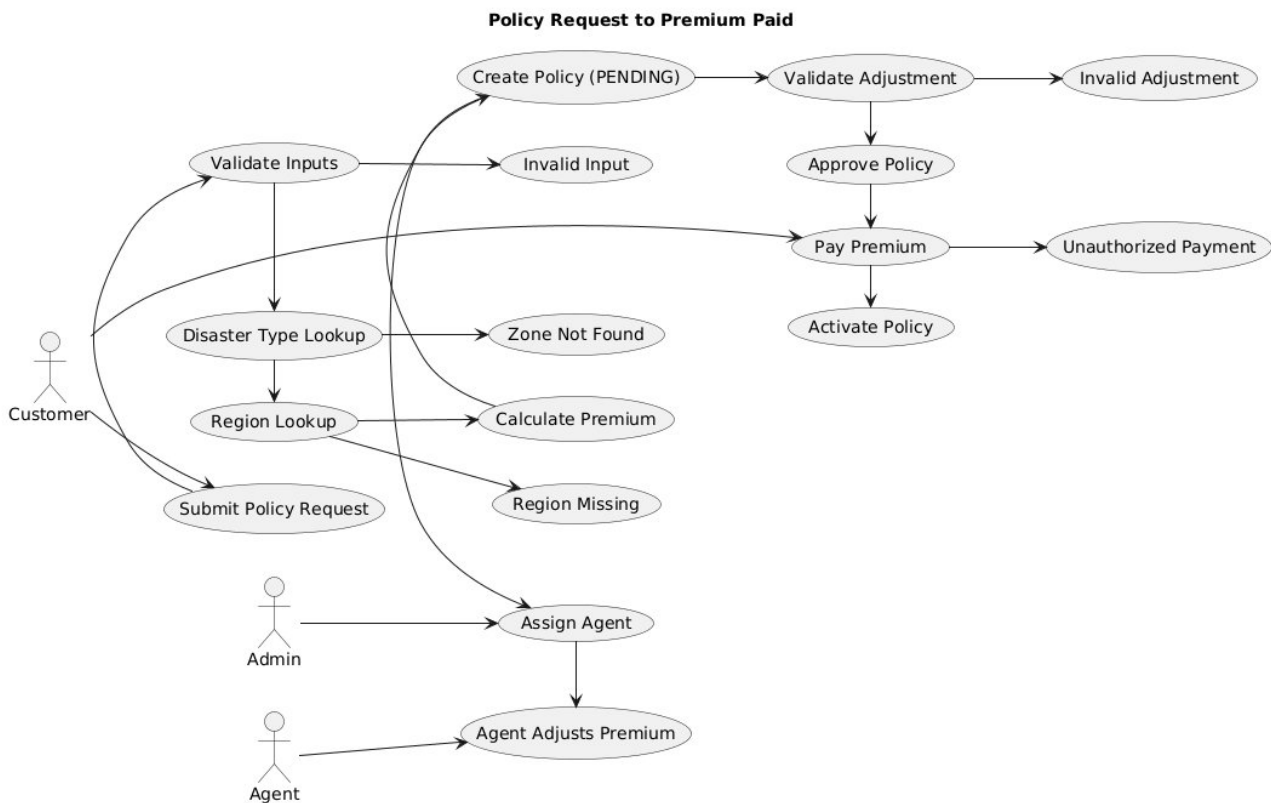


Figure 1: Use Case Diagram — Policy Request to Premium Paid

9.2 Use Case Diagram — Claim Request to Claim Paid

Illustrates the claim lifecycle from submission by the Customer through validation, assignment to a Claims Officer, investigation decision (approve/reject), payout triggering, risk pool update, and threshold check.

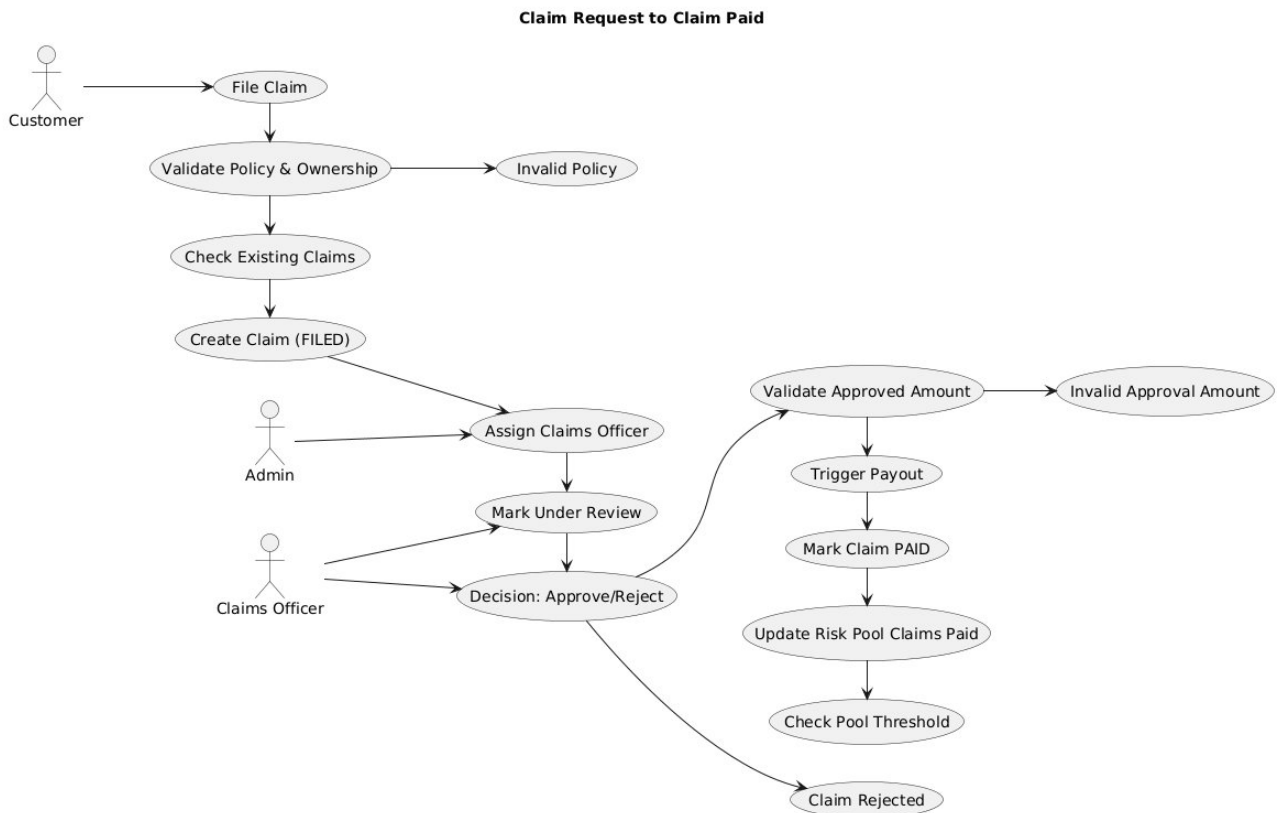


Figure 2: Use Case Diagram — Claim Request to Claim Paid

9.3 Use Case Diagram — Create Risk Pool and Disaster Zone

Illustrates the Admin's workflow for creating and configuring risk pools and disaster zones, including duplicate-prevention checks, zone-policy linkage, and automatic pool balance updates triggered by premium payments and claim payouts.

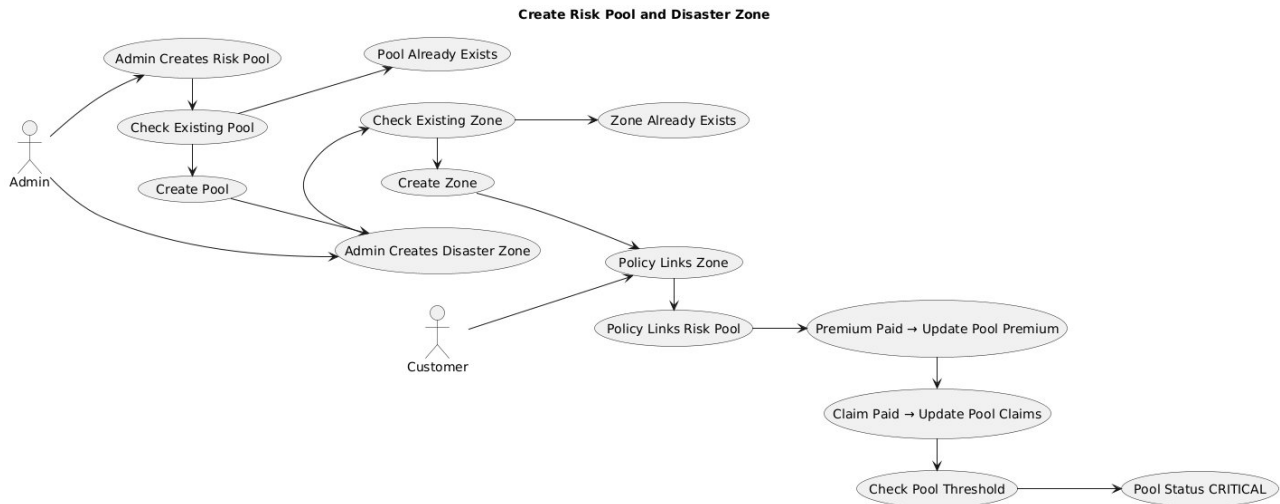


Figure 3: Use Case Diagram — Create Risk Pool and Disaster Zone

9.4 Entity-Relationship (ER) Diagram

Provides the complete database schema with a normalised relational structure showing primary keys, foreign keys, data types, and the cardinality of all entity relationships across all ten tables.

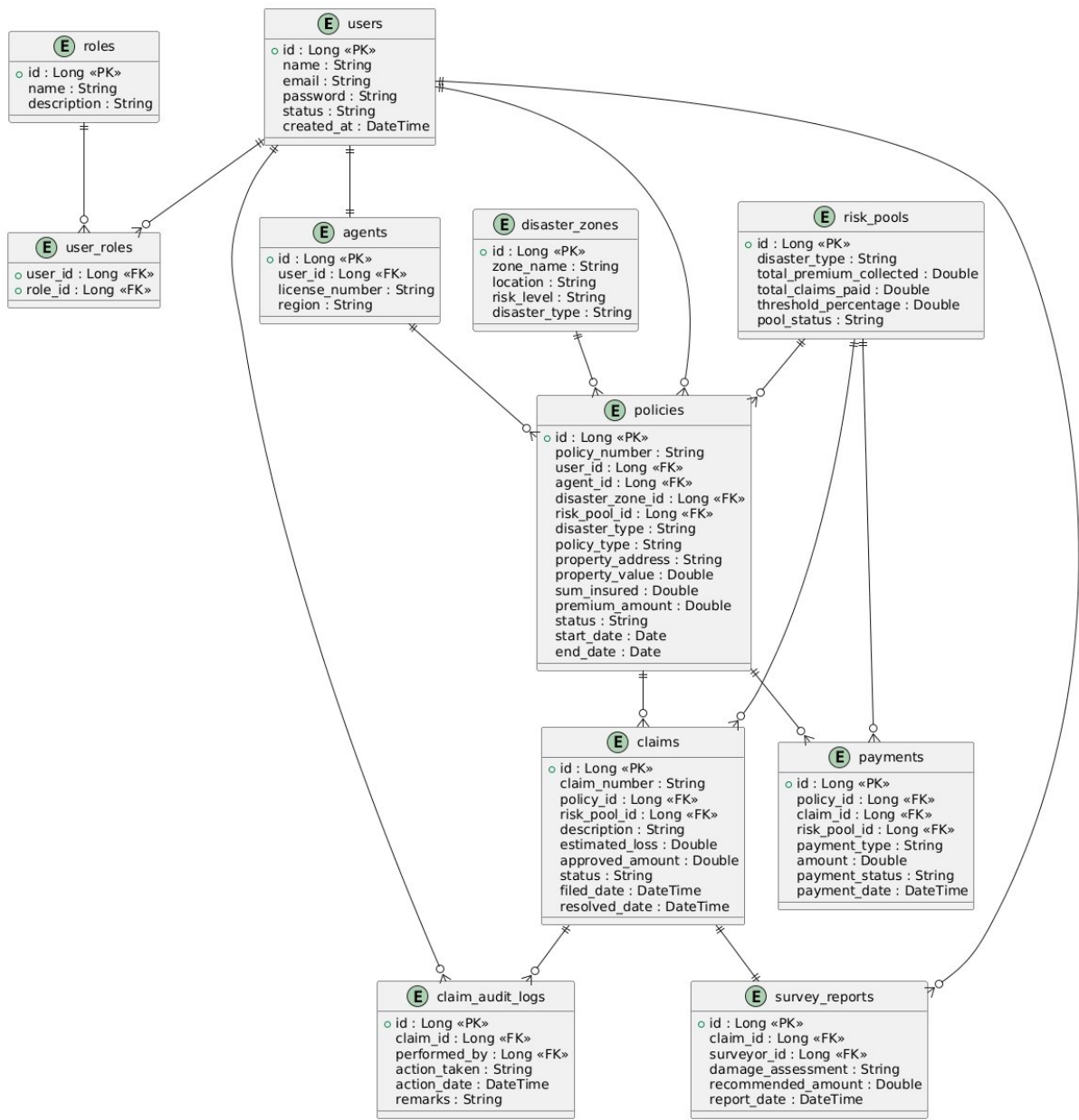


Figure 4: Entity-Relationship Diagram of the Database Schema

9.5 Activity Diagram — Pay Premium Workflow

Traces the step-by-step server-side and client-side activity flow when a Customer pays a premium, including all validation gates (user exists, policy exists, ownership check, status check, duplicate payment check), database writes, risk pool update, and UI feedback sequence.

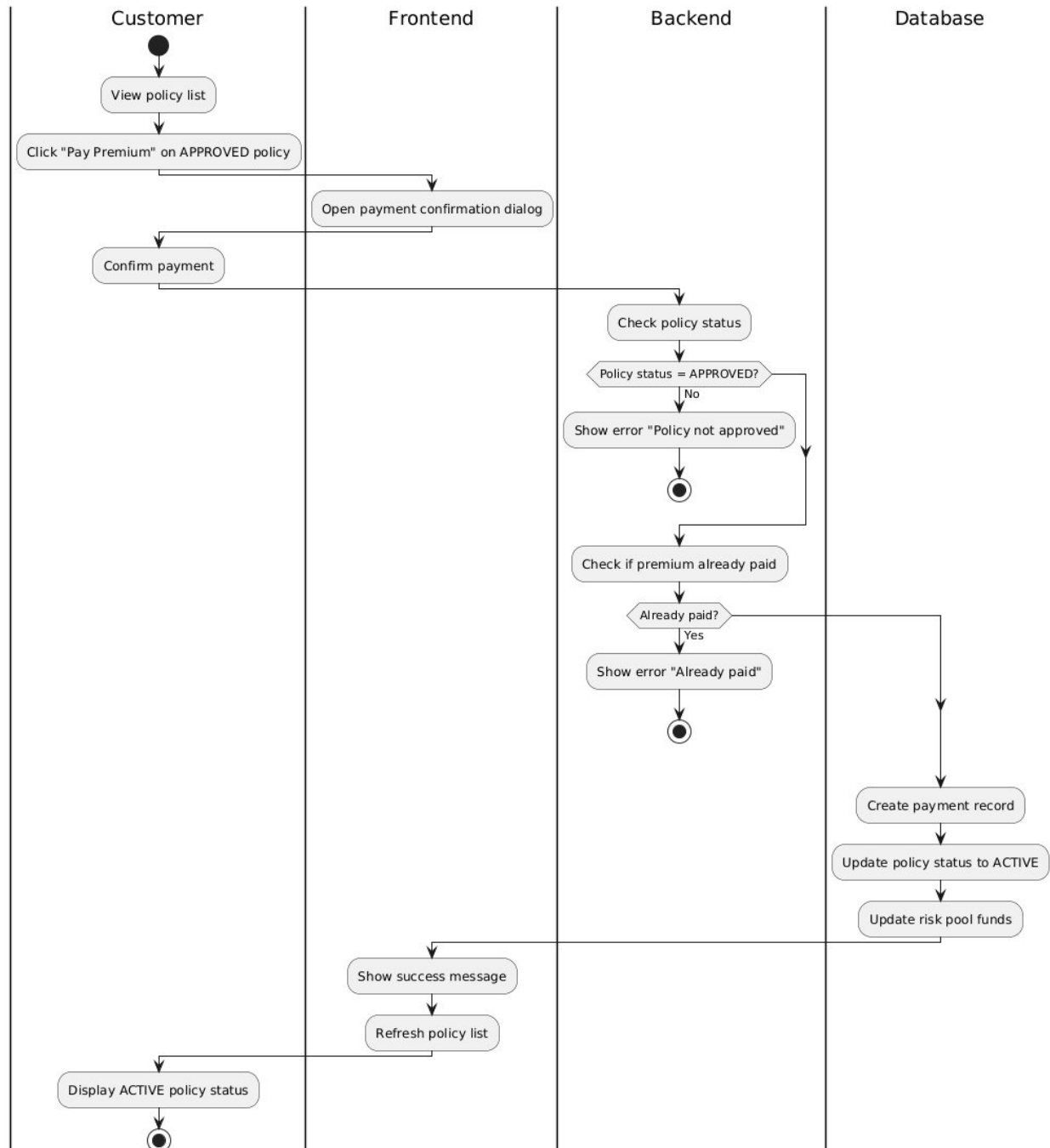


Figure 5: Activity Diagram — Pay Premium Workflow

9.6 Sequence Diagram — Pay Premium

Details the message-level interactions between Customer, Angular Frontend, Spring Boot Backend, and the Database during the premium payment use case — from the button click through payment record creation, policy status update, risk pool update, and final UI refresh.

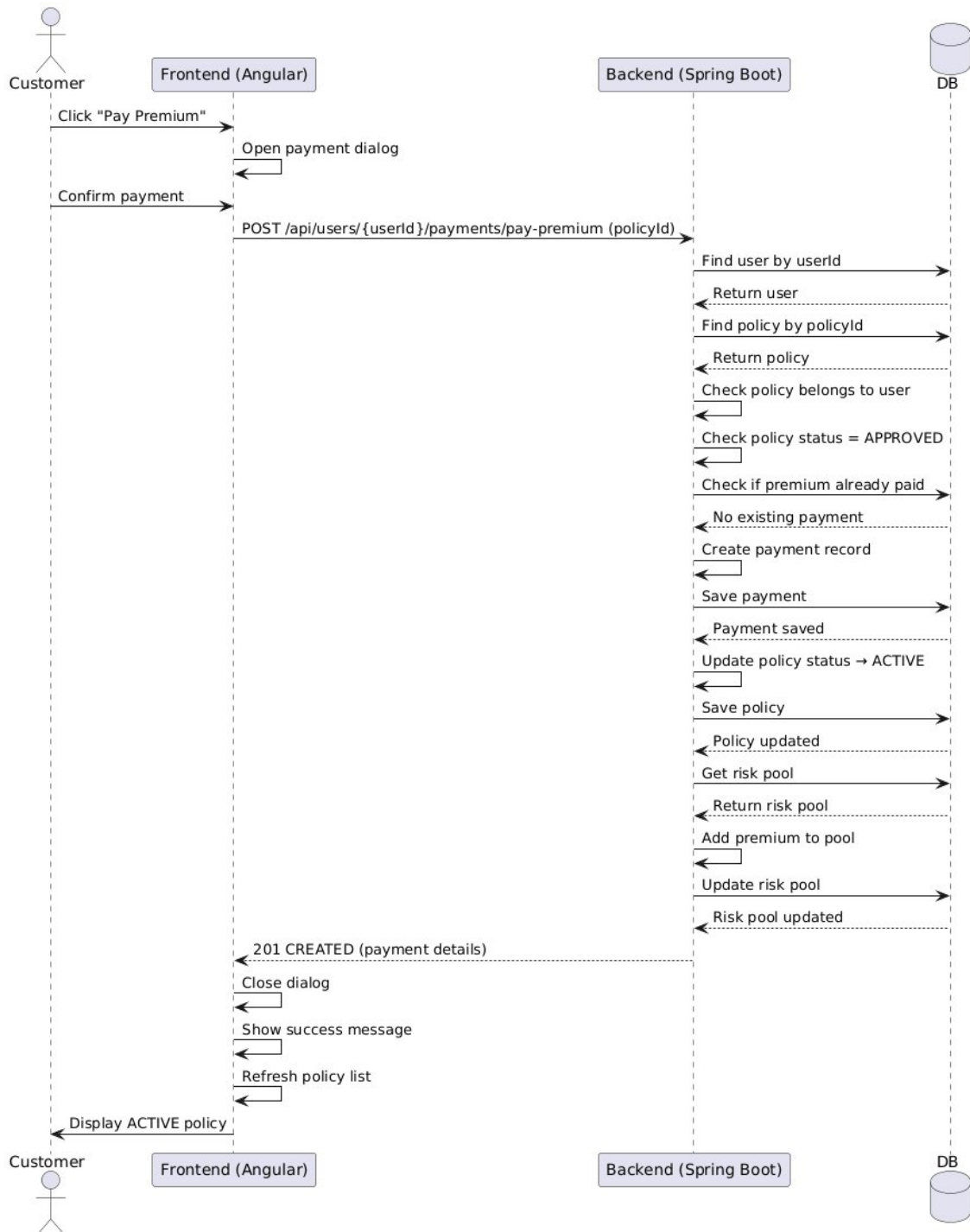


Figure 6: Sequence Diagram — Pay Premium

10. Technical Challenges & Solutions

Challenge	Problem	Solution
Multi-Role State Collisions	Managing policy and claim status transitions across roles without data inconsistency or race conditions	Strict state machine logic enforced at the service layer using enum checks. Invalid transitions return HTTP 422. @Transactional ensures atomicity.
Premium Calculation Accuracy	Floating-point arithmetic in Java producing inconsistent premium amounts across different environments	All monetary calculations use consistent double arithmetic with explicit rounding; formula validated against actuary inputs prior to release.
Risk Pool Synchronisation	Premium and claim transactions must atomically update risk pool balances to prevent financial inconsistency	Payment and pool update operations are wrapped in a single @Transactional service method; any failure triggers a full rollback.
Agent Auto-Assignment	New policies must be routed to the correct agent based on region without manual admin intervention	Agent lookup by region at policy creation time; policy is created in PENDING/unassigned state if no agent exists in the region, allowing manual assignment later.
Duplicate Payment Prevention	Concurrent requests could potentially result in multiple premium payments for the same policy	Service layer checks for an existing COMPLETED premium payment before creating a new record; duplicate attempts return HTTP 409 Conflict.
JWT Stateless Security	Stateless architecture means revoked tokens remain valid until expiry — a risk for deactivated accounts	Admin deactivation flags user as INACTIVE in DB; JwtFilter validates both token signature AND user.status == ACTIVE on every request.

11. Future Enhancements

Enhancement	Description	Priority
Email / SMS Notifications	Notify customers of policy approvals, claim updates, premium reminders, and risk pool alerts via SMTP / Twilio	High
Real Payment Gateway	Integrate Stripe or PayPal for live premium payment processing with webhook-based status confirmation	High
Document Upload	Allow customers to upload supporting photos and PDFs (images, invoices) as claim evidence, stored on filesystem	High
PDF Policy Generation	Generate cryptographically signed PDF policy documents using iText upon policy activation	Medium
Refresh Token Support	Implement refresh token mechanism to avoid forcing re-login on JWT expiry during active sessions	Medium
Advanced Reporting	Exportable PDF / Excel reports for policies, claims, financial summaries, and risk pool analytics	Medium
Survey Module	Add a dedicated Surveyor role with field inspection workflows, damage assessments, and report submission	Medium
AI-Based Fraud Detection	ML models to identify suspicious claim patterns, flag anomalous loss amounts, and reduce fraudulent payouts	Low
Mobile Application	Native iOS and Android apps for customer self-service, policy viewing, and claim submission with photo capture	Low
Government Disaster API	Real-time integration with government disaster databases to automatically update zone risk factors	Low

12. Conclusion

This Software Requirements Specification has defined the complete functional, non-functional, architectural, and data requirements for **Relief — the Disaster Insurance Management System**.

The system addresses measurable pain points in disaster insurance management: manual processing overhead, opaque claim tracking, inconsistent premium pricing, and poor financial visibility into risk pools. By replacing these with secure, automated, role-driven digital workflows, Relief delivers concrete value to both insurers and policyholders.

The architecture — built on a layered Spring Boot backend, a decoupled Angular SPA frontend, JWT-based RBAC, and JPA/Hibernate ORM — provides a maintainable and extensible foundation. The Premium Calculation Engine, Disaster Zone Risk Model, and Claims Adjudication workflow together form an automated underwriting and settlement layer that removes human error from critical financial decisions.

The six UML and workflow diagrams in Section 9 provide unambiguous visual specifications covering actor interactions, database structure, state transitions, activity flows, and message-level sequences — ensuring all implementation and testing activities can be traced directly to documented requirements.

With the future enhancements outlined in Section 11, Relief is positioned to grow into a production-ready platform capable of integrating live payment gateways, document management, survey workflows, and AI-assisted fraud detection — delivering a comprehensive, next-generation disaster insurance solution.

— END OF DOCUMENT —